AUMOVIO Data Engineering Task

Handwritten Solution Notes

Data loading • Delta loading • Database design • Transformations • SQL analysis

Almonds

→ two tools

↓ Kumquat

Provide Combined CSV containing both image and label info.

↓

its CSV combines image and label info. we must separate that info into two database tables.

Note (we have to avoid duplicate of labels for images as one image can have many labels)

↓ Papaya

Provide separate CSV files for images and labels

↓

Read CSV files and store image records in image and label records in label

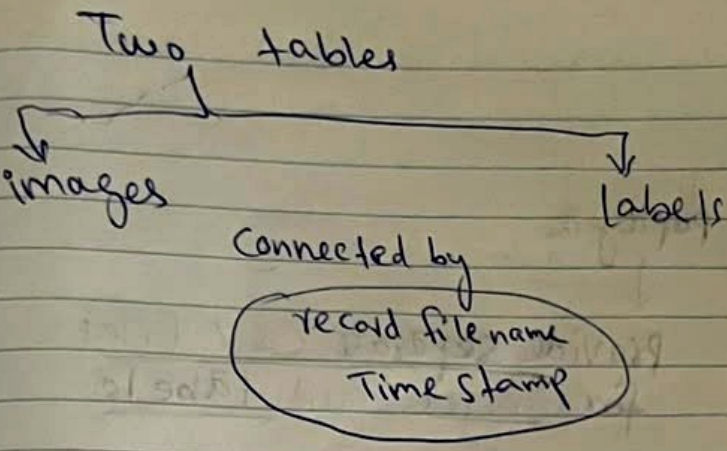
Delta loading:

→ Insert only those records that are new.

→ Keep the latest label version represents same label.

→ Map Country values. HU → Hungary

Main idea " Building the software that takes exported data from two existing labeling tools and reliably loads and maintains it in a common db.



- SQL file defines what database should look like
→ Python lets our program execute SQL and move data in/out from database

database.py → Connect to and initialize the database.

Delta loading:

Insert only new records, rather than loading records that already exist.

"IGNORE" function in database skip those records that violates a uniqueness constraints such as our primary key.

Kumquat → 13 rows
→ unique 8
→ duplicates 5

Label delta loading :-

for label identity key is { Recording name, Timestamp, label coordinates }

⇒ Avoid inserting a logical label that already exists
⇒ Keep only the highest version of each logical label.

Total labels in database = 31

13 Papaya

18 Kenya

Label version Deduplication :-

We need to keep only the row with highest version for each logical label.

Country translation :-

Hungary → Hu
Pakistan → PK

attribute map is simply a dictionary that translates one value into another.

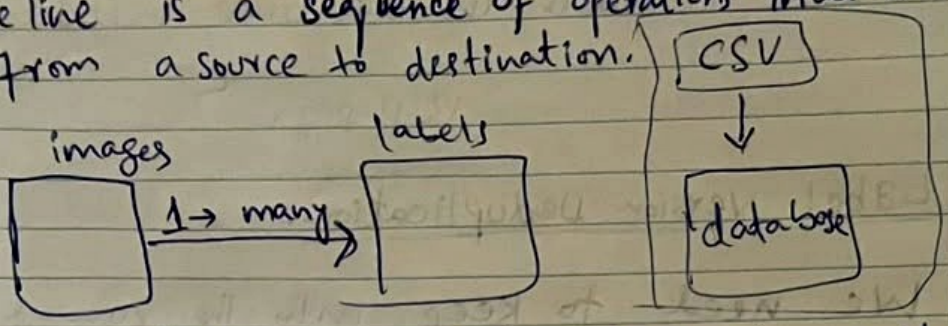
SQL Queries :-

→ Images without labels

→ find two largest labels per image

→ find image or images with the most labels whose labelsize >= 8.6 including ties

→ datapipeline is a sequence of operations that moves data from a source to destination.



So that's why we need two tables to avoid duplication.

Job: to build a small datapipeline that loads this information into a database.

⇒ database.py :- opens a connection to SQLite and initializes the database using our schema.
How python communicates with database?

⇒ loaders.py :-
Reads CSV files and insert records into database.
How does data get into the database.

⇒ transformations.py :-
Applies data transformations, including country code mapping and removing older label versions.
How we change & clean the data?

⇒ Pipeline.py :-
Coordinates the overall process, by calling all functions in a required order.
How do all the components work together?

⇒ __init__.py :- each Python file is a module.

So to make it a proper package we need to create this file so that's how it helps to execute code and to import from one file to another.

Database.py :-

- SQLite is used to store images and label records.
- Python need a way to communicate with that

two functions:
- Connect()
  - take db_path
  - opens a connection to database
- initialize_database()
  - reads our SQL schema file,
  - executes its SQL statements and commits the changes.
  - take schema_path

Loadexs.py :-

- database.py Just creates database and connection
- To put CSV data into that we need loader.
  - Read data from CSV Files
  - Load image records into image tables
  - " label " " label ".
  - Handle delta loading.

- load_csv function is to read csv files and return list of dic
- load_images function is to put it into database.
- load_labes functions are to load labels and put updated version.
- split_kumkwat_rows function is used to split image data and labels data as it contains the collected info.
- load_images_delta function is to insert only those images that thier primary key is not already overthere.
- load label image delta:- Insert new logical labels or updated existing labels whenever newer version arrives.

Pipeline.py :-

→ main idea of pipeline.py is to bring together functions from other modules.

diff

→ loading functions decide what to do with an incoming label.

→ Deduplication cleans up multiple versions that already exist in db.

remove older versions that remain in db.

Transformations.py :-

↳ deduplicate label versions:
removes older versions of logical labels

↳ map_country:-
maps country names to country codes.
- only converts name before inserting

↳ updatecountry_codes:-
update country names already stored in db.
- changes country names that are already in dB.

Testing:-

Unit_test Framework for testing our existing functions